ECE 7670
Program 4
Nathan Walker

October 9, 2025

---

1. In 5[x], determine g(x)=(2x5+3x4+4x3+3x2+ 2x+1,x4+2x3+3x2+4x+3) and also s(x) and t(x)such that a(x)s(x)+b(x)t(x)=(a(x),b(x)).

2. Compute(x3+2x2+x+4,x2+3x+4),operationsin [x], and also find polynomials s(x) and t(x) such that a(x)s(x)+b(x)t(x)=(a(x),b(x)).

3. Write a C or C++ function that performs the Euclidean algorithm on integers a and b, returning g, s, and t such that g =as+bt. The function should have declaration void gcd(int a, int b, int g, int s, int t); Test your algorithm on (24,18), (851,966), and other pairs of integers. Verify in each case that as + bt = g.

4. Write a function that computes the Euclidean algorithm on polynomialT¡TYPE¿.The function should have declaration template ¡class T¿ void gcd(const polynomialT¡T¿ a, const polynomialT¡T¿ b, polynomialT¡T¿ g, polynomialT¡T¿ s, polynomialT¡T¿ t); Also, write a program to test your function. Algorithm 5.4 shows a test program and the framework for the program, showing how to instantiate the function with ModAr and double polynomial arguments.

5. Compute (3x7 +4x6 +3x4 +x3 +1,4x4 +x3 +x) and t(x) and s(x) for polynomials in 5[x].Verify that a(x)s(x) + b(x)t(x) = g(x).

6. Compute (2x5 + 3x4 +4x3 +3x2 +2x+1,x4 + 2x3 +3x2 +4x+3)ands(x)andt(x)for polynomials in 5[x]. Verify that a(x)s(x)+b(x)t(x)=g(x).

7. Compute (2 +8x+10x2 +4x3,1+7x+14x2 +8x3) and s(x) and t(x) for polynomials in [x]. For polyno mials with real coefficients, extra care must be taken to handle roundoff. Verify that a(x)s(x)+b(x)t(x)= g(x).

8. Write a function which applies the Sugiyama algorithm to a sequence of data or its polynomial representation.

9. Test your algorithm over 5[x] by finding the shortest polynomial generating the sequence 3,2,3,1,4,0,4,3. Having found t(x), compute b(x)t(x) and identify r(x) and s(x) and verify that they are consistent with the result found by the Sugiyama algorithm.

10. In 5[x], verify that the sequence 3,2,1,0,4,3,2,1 can be generated using the polynomial t1(x)=1 + 2x + 4x2 +2x3 +x4. Then use the Sugiyama algorithm to find the shortest polynomial t(x) generating the sequence and verify that it works.

```
$ ./testpolygcd.exe
a_int=24
b_int=18
g_int=6
s_int=1
t_int=-1

a_int=966
b_int=851
g_int=23
s_int=15
t_int=-17

a_p=2x^5 + 3x^4 + 4x^3 + 3x^2 + 2x + 1
b_p=x^4 + 2x^3 + 3x^2 + 4x + 3
g_p=2
s_p=2x^3 + 2x^2 + x + 1
t_p=x^4 + 3x^3 + 3x + 2

apd=x^3 + 2x^2 + x + 4
bpd=x^2 + 3x + 4
gpd=8
spd=1
tpd=-1x + 1

a1=3x^7 + 4x^6 + 3x^4 + x^3 + 1
b1=4x^4 + x^3 + x
g1=3
s1=2x^2 + 3
t1=x^5 + 4x^4 + 3x^3 + 2x^2 + 3x

a2=2x^5 + 3x^4 + 4x^3 + 3x^2 + 2x + 1
b2=x^4 + 2x^3 + 3x^2 + 4x + 3
g2=2
s2=2x^3 + 2x^2 + x + 1
t2=x^4 + 3x^3 + 3x + 2

a3=4x^3 + 10x^2 + 8x + 2
b3=8x^3 + 14x^2 + 7x + 1
g3=3x^2 + 4.5x + 1.5
s3=1
t3=-0.5

r4=x^3 + 4x^2 + 3x + 3
s4=2x^3 + 4
t4=x^4 + 2x^3 + 4x^2 + 2x + 1
b4*t4=3x^11 + x^8 + x^3 + 4x^2 + 3x + 3

n=0, s4 = (-2*0 - 4*1 - 2*2 - 1*3) mod 5 = 4
n=1, s5 = (-2*4 - 4*0 - 2*1 - 1*2) mod 5 = 3
n=2, s6 = (-2*3 - 4*4 - 2*0 - 1*1) mod 5 = 2
n=3, s7 = (-2*2 - 4*3 - 2*4 - 1*0) mod 5 = 1
r5=x + 3
s5=4x
t5=x^2 + 3x + 1
b5*t5=x^9 + x + 3
n=0, s2 = (2*2 + 4*3) mod 5 = 1
n=1, s3 = (2*1 + 4*2) mod 5 = 0
n=2, s4 = (2*0 + 4*1) mod 5 = 4
n=3, s5 = (2*4 + 4*0) mod 5 = 3
n=4, s6 = (2*3 + 4*4) mod 5 = 2
n=5, s7 = (2*2 + 4*3) mod 5 = 1
```

Figure 1: output of the program containing all the work for problems

```cpp
#include "polynomialT.h"
// #include "ModAr.h"
#include "ModArnew.h"

#include <iostream>
using namespace std;

void gcd_int(int a, int b, int &g, int &s, int &t)
{
    int r_ = a, r0 = b, s_ = 1, s0 = 0, t_ = 0, t0 = 1;
    int ri = 1;
    int qi, si, ti;
    while(ri != 0)
    {
        qi = r_/r0;
        ri = r_ - (qi * r0);
        si = s_ - (qi*s0);
        ti = t_ - (qi * t0);
        t_ = t0;t0 = ti;
        s_ = s0;s0 = si;
        r_ = r0;r0 = ri;
    }
    s=s_;t=t_;g=r_;
}

template <class T> void gcd(const polynomialT<T> &a,const
   polynomialT<T> &b,
polynomialT<T> &g, polynomialT<T> &s, polynomialT<T> &t)
{
    polynomialT<T> r_=a,r0=b, ri=a, qi, si, ti,s__;
    polynomialT<T> s_ = polynomialT<T>(1), s0 = polynomialT<T>(0);
   polynomialT<T> t_ = polynomialT<T>(0), t0 = polynomialT<T>(1);
    while(ri != 0)
    {
        // cout << "loop bleh" << endl;
        qi = r_/r0;
        // cout << "qi = " << qi << endl;
        ri = r_ - (qi * r0);
        // cout << "ri = " << ri << endl;
        si = s_ - (qi*s0);
        ti = t_ - (qi * t0);
        t_ = t0;t0 = ti;
        s__ = s_;s_ = s0;s0 = si;
        r_ = r0;r0 = ri;
    }
    s=s_;t=t_;g=r_;
}

template <class T> void sugiyama(const polynomialT<T> &a,const
   polynomialT<T> &b,
```

```
polynomialT<T> &g, polynomialT<T> &s, polynomialT<T> &t, int p)
{
    polynomialT<T> r_=a,r0=b, ri=a, qi, si, ti,s__;
    polynomialT<T> s_ = polynomialT<T>(1), s0 = polynomialT<T>(0);
   polynomialT<T> t_ = polynomialT<T>(0), t0 = polynomialT<T>(1);
    int i = 0;
    while(ri.getdegree() >= p)
    {
        // cout << i++<<endl;
        // cout << "loop bleh" << endl;
        qi = r_/r0;
        // cout << "qi = " << qi << endl;
        ri = r_ - (qi * r0);
        // cout << "ri = " << ri << endl;
        si = s_ - (qi*s0);
        ti = t_ - (qi * t0);
        t_ = t0;t0 = ti;
        s__ = s_;s_ = s0;s0 = si;
        r_ = r0;r0 = ri;
    }
    s=si;t=ti;g=ri;
}
```

Listing 1: $\gcd_p oly.h$

```
//
//  Program: testpolygcd.cc

#include <math.h>
#include "ModArnew.h"
#include "polynomialT.cc"
#include "gcd_poly.cc"

// create instantiations of the polynomial class of type ModAr
template class polynomialT<ModAr<5> >;
template class polytemp<ModAr<5> >;
// create instantiations of the polynomial class of type double
template class polynomialT<double>;
template class polytemp<double>;

typedef ModAr<5> Z5;

// Create an instantiation of the gcd function, for polynomials of
   type ModAr
template <class Z5 > void
gcd(const polynomialT<Z5> &a, const polynomialT<Z5> &b,
    polynomialT<Z5> &g,
    polynomialT<Z5> &s, polynomialT<Z5> &t, int sdeg);

// Create an instantiation of the gcd function, for polynomials of
   type double
```

```
// template <double> void
// gcd(const polynomialT<double> &a, const polynomialT<double> &b,
//  polynomialT<double> &g,
//  polynomialT<double> &s, polynomialT<double> &t, int sdeg);


int main()
{
    // POLYC(Z5, a, {4, 0, 1, 2, 4, 0, 3, 1});
    // POLYC(Z5, b, {4, 2, 0, 3, 0, 0, 1});
    polynomialT<Z5> s_p,s1,s2,s4,s5, t_p,t1,t2,t4,t5, g_p,g1,g2,g4,
        g5;
    polynomialT<double> s3,t3,g3,spd,tpd,gpd;
    POLYC(Z5,a_p,{1,2,3,4,3,2});
    POLYC(Z5,b_p,{3,4,3,2,1});
    POLYC(double,apd,{4,1,2,1});
    POLYC(double,bpd,{4,3,1});
    POLYC(Z5,a1,{1,0,0,1,3,0,4,3});
    POLYC(Z5,b1,{0,1,0,1,4});
    POLYC(Z5,a2,{1,2,3,4,3,2});
    POLYC(Z5,b2,{3,4,3,2,1});
    POLYC(double,a3,{2,8,10,4});
    POLYC(double,b3,{1,7,14,8});
    POLYC(Z5,a4,{0,0,0,0,0,0,0,0,1});
    POLYC(Z5,b4,{3,2,3,1,4,0,4,3});
    // POLYC(Z5,a4,{0,0,0,0,0,0,1});
    // POLYC(Z5,b4,{2,3,4,2,2,3});
    int p4 = 4;
    int p5 = 4;
    POLYC(Z5,a5,{0,0,0,0,0,0,0,0,1});
    POLYC(Z5,b5,{3,2,1,0,4,3,2,1});
    int s_int,t_int,g_int,a_int,b_int;
    a1.setprintdir(1);
    a2.setprintdir(1);
    a3.setprintdir(1);

    a_int = 24;b_int=18;
    cout << "a_int="<<a_int<<endl;
    cout << "b_int="<<b_int<<endl;
    gcd_int(a_int,b_int,g_int,s_int,t_int);
    cout <<"g_int="<<g_int<<endl;
    cout<<"s_int="<<s_int<<endl;
    cout<<"t_int="<<t_int<<endl<<endl;

    a_int = 966;b_int=851;
    cout << "a_int="<<a_int<<endl;
    cout << "b_int="<<b_int<<endl;
    gcd_int(a_int,b_int,g_int,s_int,t_int);
    cout <<"g_int="<<g_int<<endl;
    cout<<"s_int="<<s_int<<endl;
    cout<<"t_int="<<t_int<<endl<<endl;
```

```
75
76      cout << "a_p=" << a_p << endl;
77     cout << "b_p=" << b_p << endl;
78     gcd(a_p,b_p,g_p,s_p,t_p);
79     cout << "g_p=" << g_p << endl;
80     cout << "s_p=" << s_p << endl;
81     cout << "t_p=" << t_p << endl;
82     cout << "\n";
83
84      cout << "apd=" << apd << endl;
85     cout << "bpd=" << bpd << endl;
86     gcd(apd,bpd,gpd,spd,tpd);
87     cout << "gpd=" << gpd << endl;
88     cout << "spd=" << spd << endl;
89     cout << "tpd=" << tpd << endl;
90     cout << "\n";
91
92
93     cout << "a1=" << a1 << endl;
94     cout << "b1=" << b1 << endl;
95     gcd(a1,b1,g1,s1,t1);
96     cout << "g1=" << g1 << endl;
97     cout << "s1=" << s1 << endl;
98     cout << "t1=" << t1 << endl;
99     cout << "\n";
100
101     cout << "a2=" << a2 << endl;
102    cout << "b2=" << b2 << endl;
103    gcd(a2,b2,g2,s2,t2);
104    cout << "g2=" << g2 << endl;
105    cout << "s2=" << s2 << endl;
106    cout << "t2=" << t2 << endl;
107    cout << "\n";
108
109     cout << "a3=" << a3 << endl;
110    cout << "b3=" << b3 << endl;
111    gcd(a3,b3,g3,s3,t3);
112    cout << "g3=" << g3 << endl;
113    cout << "s3=" << s3 << endl;
114    cout << "t3=" << t3 << endl;
115     cout<<"\n";
116
117     polynomialT<Z5>inverse4 = polynomialT<Z5>(1);
118     // cout << "a4=" << a4 << endl;
119    // cout << "b4=" << b4 << endl;
120    sugiyama(a4,b4,g4,s4,t4,p4);
121    cout << "r4=" << g4*inverse4 << endl;
122    cout << "s4=" << s4*inverse4 << endl;
123    cout << "t4=" << t4*inverse4 << endl;
124     cout << "b4*t4="<<t4*inverse4*b4<<endl<<endl;
125
```

```
126     cout << "n=0, s4 = (-2*0 - 4*1 - 2*2 - 1*3) mod 5 = 4" << endl;
127     cout << "n=1, s5 = (-2*4 - 4*0 - 2*1 - 1*2) mod 5 = 3" << endl;
128     cout << "n=2, s6 = (-2*3 - 4*4 - 2*0 - 1*1) mod 5 = 2" << endl;
129     cout << "n=3, s7 = (-2*2 - 4*3 - 2*4 - 1*0) mod 5 = 1" << endl;
130 
131     polynomialT<Z5>inverse5 = polynomialT<Z5>(1);
132     sugiyama(a5,b5,g5,s5,t5,p5);
133    cout << "r5=" << g5*inverse5 << endl;
134    cout << "s5=" << s5*inverse5 << endl;
135    cout << "t5=" << t5*inverse5 << endl;
136     cout << "b5*t5="<<t5*inverse5*b5<<endl;
137 
138     cout << "n=0, s2 = (2*2 + 4*3) mod 5 = 1" << endl;
139     cout << "n=1, s3 = (2*1 + 4*2) mod 5 = 0" << endl;
140     cout << "n=2, s4 = (2*0 + 4*1) mod 5 = 4" << endl;
141     cout << "n=3, s5 = (2*4 + 4*0) mod 5 = 3" << endl;
142     cout << "n=4, s6 = (2*3 + 4*4) mod 5 = 2" << endl;
143     cout << "n=5, s7 = (2*2 + 4*3) mod 5 = 1" << endl;
144 
145 }
146 
147 /*
148 Local Variables:
149 compile-command:"g++ -o testpolygcd -g testpolygcd.cc"
150 End:
151 */
```

Listing 2: testpolygcd.cc